FUNDAMENTALS OF DEEP LEARNING

Comprehensive Study Material

Units I & II | Complete Notes with 50 Q&A

Unit I: The Fundamentals of Deep Learning
Unit II: Gradient Descent & Neural Network Training

50 Q&A | 10-Mark | 5-Mark | 2-Mark Questions

Table of Contents

UNIT I: FUNDAMENTALS OF DEEP LEARNING

1.1 Deep Learning vs Machine Learning
1.2 Evolution of AI and ML — Historical Epochs
1.3 Deep Learning Relevance Today
1.4 The Matrix Magic: Scalars, Vectors, Matrices, Tensors
1.5 Real-World Data Representation with Tensors
1.6 Tensor Operations and Visualisation
1.7 Visualising Deep Learning — The Big Picture

UNIT II: GRADIENT DESCENT & NEURAL NETWORK TRAINING

2.1 Derivatives and Tensors
2.2 Gradient Descent and Stochastic Gradient Descent
2.3 Backpropagation — The Chain Rule
2.4 Layers: The Atoms of Deep Learning
2.5 Models: The Molecules of Deep Learning
2.6 Loss Functions
2.7 Optimizers
2.8 Activation Functions
2.9 Deep Learning in Action: Classification & Regression
2.10 Overfitting and Underfitting

Q&A SECTION

Section A: 10-Mark Questions (Q1–Q15)
Section B: 5-Mark Questions (Q16–Q35)
Section C: 2-Mark Questions (Q36–Q50)

UNIT I: The Fundamentals of Deep Learning

1.1 Deep Learning vs Machine Learning

Machine Learning (ML) is a subset of Artificial Intelligence that enables systems to learn from data. Deep Learning (DL) is a further subset of ML that uses multi-layered artificial neural networks to model complex patterns. Understanding the distinction is foundational.

Machine Learning	Deep Learning
Requires manual feature engineering	Learns features automatically from raw data
Works well with smaller datasets	Requires large datasets to excel
Interpretable (decision trees, linear models)	Often a 'black box' — less interpretable
Lower compute requirements	High compute: GPUs/TPUs essential
Random Forest, SVM, XGBoost	CNNs, RNNs, Transformers, GANs
Tabular, structured data excels	Images, speech, text, video
Performance plateaus with more data	Performance scales with more data

The key philosophical difference: traditional ML separates feature extraction (human-designed) from learning (algorithmic). Deep learning collapses these into a single end-to-end trainable pipeline where the network itself discovers which features are useful.

1.2 Evolution of AI and ML — Historical Epochs

The journey of AI spans nearly a century, marked by cycles of optimism, disillusionment (AI winters), and breakthrough resurgences.

1.2.1 The Symbolic AI Era (1950s–1980s)

Early AI focused on rule-based systems and logic. Alan Turing proposed the Turing Test (1950). Researchers believed intelligence could be captured through explicit rules and symbol manipulation. The General Problem Solver (GPS) and expert systems like MYCIN emerged. Limitation: brittle — could not handle real-world noise and ambiguity.

1.2.2 The First AI Winter (1974–1980) & Second (1987–1993)

Overpromising led to funding cuts. The perceptron (Rosenblatt, 1958) was shown limited by Minsky & Papert (1969). Without enough compute and data, neural networks stalled.

1.2.3 The Statistical ML Revolution (1990s–2000s)

SVMs, decision trees, boosting, and probabilistic models dominated. Data-driven approaches replaced hand-crafted rules. The internet began generating vast datasets.

1.2.4 The Deep Learning Renaissance (2006–present)

- 2006: Hinton's Deep Belief Networks — efficient layer-wise pretraining
- 2012: AlexNet wins ImageNet by a 10%+ margin — GPU-powered deep CNNs go mainstream
- 2014: GANs introduced (Goodfellow et al.) — generative modelling revolution

- 2017: Transformer architecture (Vaswani et al.) — attention is all you need
- 2018–2023: BERT, GPT-2/3/4, LLaMA, Stable Diffusion — foundation model era
- 2023–present: Multimodal AI, RLHF-tuned assistants, autonomous agents

Why Now? The Three Drivers of Deep Learning

1. DATA: The internet, smartphones, sensors, and digital services generate billions of labelled examples daily.2. COMPUTE: NVIDIA GPUs provide thousands of parallel CUDA cores; TPUs offer 100+ TFLOPS. Cloud computing democratizes access.3. ALGORITHMS: Better architectures (ResNets, Transformers), optimizers (Adam), regularization (Dropout, BatchNorm), and open-source frameworks (TensorFlow, PyTorch) accelerate progress.

1.3 Deep Learning Relevance Today

Deep learning has permeated virtually every domain of science and industry. Its relevance stems from its ability to process raw, unstructured data at scale — eliminating years of domain-specific feature engineering.

- Computer Vision: Object detection (YOLO), image segmentation (U-Net), face recognition
- Natural Language Processing: Machine translation, summarisation, question answering (GPT-4, Claude)
- Speech: Voice recognition (Whisper), text-to-speech (WaveNet), music generation
- Healthcare: Cancer detection from imaging, drug discovery (AlphaFold for protein folding)
- Autonomous Systems: Self-driving cars (Tesla FSD), robotics, drone navigation
- Generative AI: Image synthesis (Stable Diffusion), code generation (Copilot), video (Sora)
- Scientific Research: Climate modelling, materials discovery, particle physics

The 'elephant in the room' in deep learning is interpretability: despite extraordinary performance, we often do not fully understand why neural networks make specific decisions — a major concern for safety-critical applications.

1.4 The Matrix Magic: Scalars, Vectors, Matrices, Tensors

Mathematics of deep learning is built on a hierarchy of data structures. Understanding this hierarchy is essential for implementing and reasoning about neural networks.

1.4.1 Scalars

A scalar is a single numerical value — a 0-dimensional tensor. It has no direction, only magnitude. Examples: a learning rate $\eta = 0.001$, a loss value $L = 0.47$, temperature $T = 37.0$.

Scalar Notation

$x \in \mathbb{R}$ (a real number)Examples: $x = 5$, $\pi \approx 3.14159$, $e \approx 2.71828$In Python/NumPy: x = np.float32(5.0)

1.4.2 Vectors

A vector is a 1-dimensional ordered array of scalars. It has both magnitude and direction. Geometrically, it represents a point or direction in n-dimensional space. A feature vector represents one data example; a weight vector represents a neuron's parameters.

Vector Notation & Operations

$x = [x1, x2, ..., xn]^T \in \mathbb{R}^n$ (column vector)Addition: $x + y = [x1+y1, ..., xn+yn]$Dot product: $x \cdot y = \Sigma xi*yi = ||x|| \, ||y|| \cos(\theta)$L2 norm: $||x|| = \text{sqrt}(\Sigma xi^2)$In NumPy: x = np.array([1, 2, 3])

1.4.3 Matrices

A matrix is a 2-dimensional rectangular array of scalars arranged in rows and columns. In neural networks, weight matrices connect layers. A dataset of n examples with p features is an n×p matrix.

Matrix Notation & Operations

$A \in \mathbb{R}^{(m\times n)}$ — m rows, n columnsMatrix multiplication: $C = AB$, where $Cij = \Sigma k \, Aik*Bkj$ (requires A: m×k, B: k×n → C: m×n)Transpose: A^T, where $(A^T)ij = Aji$Element-wise (Hadamard): $A \odot B$ (same shape)In NumPy: A = np.array([[1,2],[3,4]]); C = A @ B

1.4.4 Tensors

A tensor is a generalisation of scalars, vectors, and matrices to arbitrary dimensions (rank). The rank (order) of a tensor is the number of dimensions. Tensors are the fundamental data structure of deep learning frameworks.

Tensor Rank	Description	Shape Example	Real-World Example
Rank 0 (Scalar)	Single number	( )	Loss value: 0.32
Rank 1 (Vector)	1D array	(n,)	Feature vector: (784,)
Rank 2 (Matrix)	2D grid	(m, n)	Weight matrix: (128, 64)
Rank 3	3D block	(batch, rows, cols)	Grayscale images: (32, 28, 28)
Rank 4	4D block	(batch, H, W, C)	RGB images: (32, 224, 224, 3)
Rank 5	5D block	(batch, time, H, W, C)	Video batch: (8, 16, 112, 112, 3)

In PyTorch: tensors = torch.tensor(...). GPU tensors: tensor.to('cuda'). Key operations: reshape, transpose, squeeze, unsqueeze, concatenate, stack.

1.5 Real-World Data Representation with Tensors

Every type of real-world data can be represented as tensors, enabling uniform processing by deep learning models.

1.5.1 Images

A grayscale image of height H and width W is a rank-2 tensor of shape (H, W). A colour (RGB) image is rank-3 of shape (H, W, 3). A batch of N colour images is rank-4 of shape (N, H, W, 3) or (N, 3, H, W) in PyTorch (channels-first). Pixel values are typically normalised to [0, 1] or [-1, 1].

1.5.2 Text

Words are mapped to integer indices via a vocabulary. A sentence of length T is a rank-1 tensor of shape (T,). A batch of B sentences of length T is (B, T). Word embeddings map integers to dense vectors of dimension d, producing a (B, T, d) rank-3 tensor.

1.5.3 Audio

A 1-second mono audio signal sampled at 16 kHz = 16,000 scalar values → rank-1 tensor (16000,). Spectrogram representations convert audio to (time_frames, frequency_bins) 2D tensors, suitable for CNN processing.

1.5.4 Tabular Data

N examples with p features → rank-2 tensor (N, p). Each row is a feature vector. Categorical features are one-hot encoded or embedded. Time-series tabular data adds a time dimension → (N, T, p).

1.6 Tensor Operations and Visualisation

Deep learning is fundamentally a sequence of tensor operations. Every forward pass through a neural network consists of matrix multiplications, element-wise activations, and reductions.

1.6.1 Core Tensor Operations

- Element-wise operations: addition (+), subtraction (-), multiplication (*), division (/), power — broadcast across shapes
- Matrix multiplication (matmul/@): Core of fully connected and attention layers
- Reduction operations: sum, mean, max, min — collapse dimensions (e.g., global average pooling)
- Reshaping: flatten (N,H,W,C) → (N, H*W*C); view/reshape without copying data
- Slicing/indexing: tensor[0], tensor[:, 2:5], tensor[mask] — fancy indexing
- Broadcasting: tensors of incompatible shapes can be combined if dimensions align (e.g., (3,1) + (1,4) → (3,4))

1.6.2 Broadcasting Rules

NumPy/PyTorch Broadcasting

Rule: Shapes are compared right-to-left. Dimensions are compatible if equal or one of them is 1.Example: A.shape=(3,4), B.shape=(4,) → B broadcast to (3,4)Example: A.shape=(5,1,3), B.shape=(1,4,3) → result shape (5,4,3)Used heavily in batch processing: bias vector (d,) + activations (batch, d) → (batch, d)

1.6.3 Visualising Deep Learning

Visualisation tools are essential for understanding what neural networks learn. Common techniques:

- TensorBoard: Track loss, accuracy, gradient norms across training steps. View model graph.

- Feature map visualisation: Display intermediate CNN layer activations to see what patterns are detected
- Filter visualisation: Visualise learned convolutional filters (first layers learn edges, later layers learn complex objects)
- t-SNE / UMAP: Reduce high-dimensional embeddings to 2D — visualise clustering of learned representations
- Gradient-weighted Class Activation Maps (Grad-CAM): Highlight image regions important for a CNN's decision
- Loss landscape visualisation: Plot 2D/3D slices of the loss surface to understand optimisation geometry

1.7 The Elephant in the Room — Interpretability & Trust

The phrase 'elephant in the room' in deep learning refers to interpretability: these models achieve superhuman performance on many tasks, yet remain largely opaque. We cannot easily explain why a specific prediction is made. This has serious consequences for deployment in healthcare, law, finance, and safety-critical systems.

- Black-box problem: Millions of parameters interact non-linearly — no simple rule explains outputs
- Adversarial vulnerability: Tiny imperceptible perturbations can cause confident wrong predictions
- Dataset bias: Models memorise and amplify biases present in training data
- Hallucination: Language models produce plausible-sounding but factually incorrect outputs
- Approaches: LIME (Local Interpretable Model-agnostic Explanations), SHAP values, attention visualisation, probing classifiers, mechanistic interpretability

UNIT II: Gradient Descent & Neural Network Training

2.1 Derivatives and Tensors

Derivatives are the mathematical engine of deep learning. Training a neural network means finding parameters that minimise a loss function — and derivatives tell us which direction to move in parameter space.

2.1.1 The Derivative

The derivative of f(x) at a point measures the rate of change: f'(x) = lim(h→0) [f(x+h) - f(x)] / h. It is the slope of the tangent line. For training: if dL/dw > 0, the loss increases as w increases, so we should decrease w. If dL/dw < 0, we should increase w.

2.1.2 Partial Derivatives and Gradients

For a function f(x1, x2, ..., xn) with multiple inputs, the partial derivative ∂f/∂xi measures the effect of xi alone. The gradient ∇f collects all partial derivatives into a vector: ∇f = [∂f/∂x1, ∂f/∂x2, ..., ∂f/∂xn]. The gradient always points in the direction of steepest ascent.

Gradient of Loss w.r.t Parameters

For loss L(W) where W is the weight matrix:∇_W L = [∂L/∂w11, ∂L/∂w12, ..., ∂L/∂wmn] — same shape as WThis is a matrix (rank-2 tensor) of partial derivatives.For a batch: gradients are averaged over the batch dimension.

2.1.3 Jacobians and Hessians

The Jacobian J is a matrix of first-order partial derivatives for vector-valued functions: Jij = ∂fi/∂xj. The Hessian H is a matrix of second-order derivatives: Hij = ∂²L/∂xi∂xj. Deep learning primarily uses first-order gradient methods (the Hessian is too expensive to compute for millions of parameters).

2.2 Gradient Descent and Stochastic Gradient Descent

Gradient descent is the core optimisation algorithm for training neural networks. It iteratively updates parameters in the direction that reduces the loss.

2.2.1 Batch Gradient Descent

Batch Gradient Descent Update Rule

θ ← θ - η * ∇_θ L(θ; X, Y)θ: all model parametersη: learning rate (step size) — critical hyperparameterL(θ; X, Y): loss computed over the ENTIRE training datasetProblem: Computing the gradient over millions of examples per step is computationally prohibitive.

2.2.2 Stochastic Gradient Descent (SGD)

SGD updates parameters using the gradient from a single random training example per step. This is noisy but much faster — can perform thousands of updates while batch GD performs one.

SGD Update Rule

For each training example (xi, yi) randomly sampled:$\theta \leftarrow \theta - \eta * \nabla_\theta L(\theta; x_i, y_i)$Advantages: Very fast updates, can escape local minima (noise helps), enables online learning.Disadvantage: High variance — loss oscillates significantly.

2.2.3 Mini-Batch Gradient Descent (Standard Practice)

Mini-batch SGD computes gradients on a small batch B of examples (typically 32–512). This balances the efficiency of vectorised computation with the regularisation effect of gradient noise.

Mini-Batch SGD Update

$\theta \leftarrow \theta - \eta * (1/|B|) * \Sigma_{i \in B} \nabla_\theta L(\theta; x_i, y_i)$Batch size effects:- Larger batch: smoother gradients, faster per-step (GPU utilisation), less noise → may converge to sharp minima- Smaller batch: noisier, more updates per epoch, more regularisation → tends to find flat minima (better generalisation)- Common values: 32, 64, 128, 256

2.2.4 The Learning Rate

The learning rate η is the most critical hyperparameter. Too large → loss diverges (overshoots minimum). Too small → training is extremely slow and may get stuck. Learning rate schedules adaptively reduce η during training:

- Step decay: Reduce by factor γ every k epochs: $\eta_t = \eta_0 * \gamma^{(t/k)}$
- Cosine annealing: $\eta_t = \eta_{min} + (\eta_{max} - \eta_{min}) * (1 + \cos(\pi t/T)) / 2$
- Warm-up: Start with small η, increase for initial epochs, then decay
- Cyclical LR (CLR): Oscillate between bounds — helps escape saddle points

2.3 Backpropagation — The Chain Rule of Differentiation

Backpropagation is the algorithm for efficiently computing gradients of the loss with respect to all parameters in a deep network. It applies the chain rule of calculus layer by layer, propagating gradients from output back to input.

2.3.1 The Chain Rule

Chain Rule

For composite function: $L = f(g(x))(dL/dx) = (dL/dg) * (dg/dx)$For a deep network with layers: $L = f_n(f_{n-1}(...f_1(x)...))\partial L/\partial x = (\partial L/\partial a_n) * (\partial a_n/\partial a_{n-1}) * ... * (\partial a_2/\partial a_1) * (\partial a_1/\partial x)$where a_l is the activation of layer l.

2.3.2 Forward Pass

During the forward pass, inputs are propagated layer by layer: $a_0 = x$ (input); $z_l = W_l * a_{l-1} + b_l$ (pre-activation); $a_l = \sigma(z_l)$ (post-activation via activation function σ). All intermediate values are stored in memory — they are needed in the backward pass.

2.3.3 Backward Pass

Starting from the loss L, we propagate gradients backwards:

Backpropagation Equations (single layer)

Output layer: $\delta_L = \partial L/\partial z_L = \partial L/\partial a_L \odot \sigma'(z_L)$Hidden layer l: $\delta_l = (W_{l+1}^T \delta_{l+1}) \odot \sigma'(z_l)$Gradients: $\partial L/\partial W_l = \delta_l * a_{l-1}^T$ $\partial L/\partial b_l = \delta_l$$\delta_l$: error signal at layer l$\sigma'(z_l)$: derivative of activation function at layer l$\odot$: element-wise (Hadamard) product

2.3.4 Computational Graphs

Modern frameworks (PyTorch, TensorFlow) represent computations as directed acyclic graphs (DAGs). Each node is an operation; edges carry tensors. Autograd automatically builds this graph during the forward pass and traverses it backwards to compute gradients (reverse-mode automatic differentiation).

PyTorch example: loss.backward() triggers autograd to compute all gradients. Each parameter tensor's .grad attribute stores its gradient after the backward pass.

2.4 Layers: The Atoms of Deep Learning

If a neural network is a molecule, layers are its atoms — the fundamental building blocks. Each layer transforms its input tensor into an output tensor via learned parameters.

2.4.1 Dense (Fully Connected) Layer

Every neuron in the layer is connected to every neuron in the previous layer. Computation: output = $\sigma(W * input + b)$, where $W \in \mathbb{R}^{out \times in}$, $b \in \mathbb{R}^{out}$. Captures global relationships. Computationally expensive for high-dimensional inputs (e.g., images).

2.4.2 Convolutional Layer

Applies learnable filters over local regions of the input. Weight sharing across spatial positions reduces parameters dramatically. Exploits translational invariance. Output size: $\text{floor}((W - F + 2P)/S) + 1$. Stacking conv layers creates hierarchical feature detectors.

2.4.3 Pooling Layer

Max pooling: takes the maximum value in each window — retains strongest activation, provides spatial invariance. Average pooling: computes mean — smoother downsampling. Global Average Pooling (GAP): reduces each feature map to a single value — used in modern CNNs before the classification head.

2.4.4 Recurrent Layer (RNN / LSTM / GRU)

Processes sequential data by maintaining a hidden state. RNN: $h_t = \tanh(W_h * h_{t-1} + W_x * x_t + b)$. LSTM adds forget, input, and output gates to address vanishing gradients. GRU simplifies LSTM with reset and update gates.

2.4.5 Embedding Layer

Converts integer indices to dense vectors. A lookup table of shape (vocab_size, embedding_dim). Learnable — the model learns meaningful representations. The bridge between discrete tokens and continuous neural network computations.

2.4.6 Normalisation Layers

Batch Normalisation: normalises per feature across batch dimension — accelerates training, acts as regulariser. Layer Normalisation: normalises per sample across feature dimension — better for small

batches and Transformers. Group Normalisation: normalises across groups of channels — useful for detection tasks with small batch sizes.

2.4.7 Dropout Layer

Randomly zeros activations with probability p during training. Forces robustness and prevents co-adaptation. Equivalent to ensemble of 2^n networks. At inference: all neurons active, activations scaled by (1-p).

2.5 Models: The Molecules of Deep Learning

A model is a composition of layers forming a complete architecture. Just as atoms bond into molecules, layers connect into models that accomplish specific tasks.

2.5.1 Sequential Models

The simplest architecture: a linear stack of layers where each layer has one input and one output. Used for simple MLPs, basic CNNs, RNNs. In Keras: model = tf.keras.Sequential([...layers...]). Limitation: cannot express branching or skip connections.

2.5.2 Functional API / DAG Models

Allows arbitrary directed acyclic graphs: skip connections (ResNet), multi-input/output, branching (Inception), shared layers. More flexible — the standard for research architectures. PyTorch uses nn.Module subclassing, enabling arbitrary Python code in forward().

2.5.3 The Training Loop

Standard Training Loop (PyTorch)

for epoch in range(num_epochs): for batch_X, batch_y in dataloader: optimizer.zero_grad() # clear previous gradients predictions = model(batch_X) # forward pass loss = criterion(predictions, batch_y) # compute loss loss.backward() # backward pass (backprop) optimizer.step() # update parameters validate(model, val_loader) # check generalisation

2.6 Loss Functions

The loss function (cost function, objective function) measures the discrepancy between the model's predictions and the true labels. The training process minimises this function. Choosing the right loss function is crucial — it defines what 'good' means for the model.

2.6.1 Regression Losses

Mean Squared Error (MSE)

MSE = (1/n) Σ (yi - ŷi)²Squares the error → heavily penalises large errors (outlier-sensitive)Gradient: ∂MSE/∂ŷi = -2(yi - ŷi)/n (smooth, differentiable everywhere)Use: Standard regression, when outliers are rareAlso called L2 Loss

Mean Absolute Error (MAE) / L1 Loss

MAE = (1/n) Σ |yi - ŷi|Linear penalty → robust to outliersGradient: ±1/n (constant, not differentiable at 0 — use subgradient)Use: Regression with outliersHuber Loss combines MSE (for small errors) and MAE (for large errors): robust + smooth

2.6.2 Classification Losses

Binary Cross-Entropy (BCE)

BCE = -(1/n) Σ [yi*log(ŷi) + (1-yi)*log(1-ŷi)]yi ∈ {0,1}, ŷi ∈ (0,1) from sigmoid outputMeasures surprise/information content — minimised when ŷ = yGradient: ∂BCE/∂ŷi = (ŷi - yi) / (ŷi(1-ŷi)) — clean gradient signalUse: Binary classification, multi-label classification

Categorical Cross-Entropy (CCE)

CCE = -(1/n) Σ_i Σ_k y_{ik} * log(ŷ_{ik})y_{ik}: one-hot encoded true label (1 for correct class, 0 else)ŷ_{ik}: softmax probability for class kFor single correct label: CCE = -(1/n) Σ log(ŷ_{i,correct})Sparse CCE: takes integer class labels directly (equivalent, more memory efficient)KL Divergence: measures distribution difference — generalises cross-entropy

2.7 Optimizers

Optimisers are algorithms that use gradients to update model parameters. They differ in how they use gradient history, adapt learning rates, and handle the geometry of the loss landscape.

2.7.1 Vanilla SGD with Momentum

SGD with Momentum

velocity: v_t = β*v_{t-1} + (1-β)*g_t (β typically 0.9)update: θ_t = θ_{t-1} - η*v_tMomentum accumulates a velocity vector in directions of persistent gradient descent.Dampens oscillations in high-curvature directions, accelerates along ravines.Nesterov momentum: compute gradient at lookahead position θ - η*β*v for better convergence.

2.7.2 AdaGrad

Adapts the learning rate per parameter: parameters that receive large/frequent gradients get smaller effective learning rates; rare parameters get larger rates. G_t += g_t²; θ_t = θ_{t-1} - (η/√(G_t + ε)) * g_t. Good for sparse data (NLP). Problem: monotonically decreasing LR — learning can stop prematurely.

2.7.3 RMSProp

RMSProp

s_t = ρ*s_{t-1} + (1-ρ)*g_t² (exponential moving average of squared gradients)θ_t = θ_{t-1} - (η / sqrt(s_t + ε)) * g_tρ typically 0.9 — prevents monotonic decrease of AdaGradGood for non-stationary problems (RNNs, online learning)

2.7.4 Adam (Adaptive Moment Estimation)

Adam — The Standard Deep Learning Optimizer

$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ (1st moment: mean of gradients, $\beta_1=0.9$)$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ (2nd moment: uncentered variance, $\beta_2=0.999$)$\hat{m}_t = m_t / (1-\beta_1^t)$ (bias correction — important at start)$\hat{v}_t = v_t / (1-\beta_2^t)$$\theta_t = \theta_{t-1} - \eta * \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$ ($\varepsilon=1e-8$)Combines momentum (1st moment) + per-parameter adaptive LR (2nd moment)Default: $\eta=0.001$, $\beta_1=0.9$, $\beta_2=0.999$ — usually works well out of the boxAdamW: Adam + decoupled L2 weight decay — better generalisation, standard for Transformers

Optimizer	Adaptive LR	Momentum	Best For
SGD	No	Optional	CNNs (with tuning)
SGD + Momentum	No	Yes	Image classification
AdaGrad	Yes (cumulative)	No	Sparse NLP features
RMSProp	Yes (EMA)	No	RNNs, non-stationary
Adam	Yes (EMA)	Yes	General purpose (default)
AdamW	Yes (EMA)	Yes	Transformers, LLMs

2.8 Activation Functions

Activation functions introduce non-linearity into neural networks. Without them, any deep network collapses to a single linear transformation. Choosing the right activation function affects gradient flow, convergence speed, and model expressiveness.

2.8.1 Sigmoid and Tanh

Sigmoid & Tanh

Sigmoid: $\sigma(x) = 1 / (1 + e^{-x})$ → output $\in (0, 1)$Derivative: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ → max 0.25 at $x=0$ → VANISHING GRADIENTS for $|x|>3$Not zero-centered → slow convergence (all-positive gradients)Use: Output layer for binary classification ONLYTanh: $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ → output $\in (-1, 1)$Derivative: $1 - \tanh^2(x)$ → max 1 at $x=0$, still saturatesZero-centered (better than sigmoid)Use: LSTM/GRU gates, some RNN hidden states

2.8.2 ReLU and Variants

ReLU Family

ReLU: $f(x) = \max(0, x)$ → output $\in [0, \infty)$Derivative: 1 if $x>0$, else 0 — No saturation for positive inputs → fast convergenceProblem: Dying ReLU — neurons with $x<0$ always output 0, gradient=0, never recoverLeaky ReLU: $f(x) = x$ if $x>0$, else αx (α~0.01) → fixes dying ReLUParametric ReLU (PReLU): α is learnableELU: $f(x) = x$ if $x>0$, else $\alpha(e^x - 1)$ → smooth negative, zero-centeredSELU: self-normalising — activations maintain unit mean/variance under certain init

2.8.3 GELU and Modern Activations

GELU (Gaussian Error Linear Unit)

GELU(x) = x * Φ(x) = x * (1/2)[1 + erf(x/√2)]Approximation: GELU(x) ≈ 0.5x(1 + tanh(√(2/π)(x + 0.044715x³)))Smooth, differentiable everywhere — no dying neuronsStochastic interpretation: randomly gates inputs by value magnitudeStandard in BERT, GPT, most modern TransformersSiLU/Swish: x * sigmoid(x) — similar performance, slightly smootherMish: x * tanh(softplus(x)) — unbounded above, bounded below

2.8.4 Softmax — For Multiclass Output

Softmax Function

softmax(z)_k = e^(z_k) / Σ_j e^(z_j) → output ∈ (0,1), all outputs sum to 1Converts raw logits to probability distribution over K classesNumerically stable implementation: subtract max(z) before exponentiatingUsed as the final layer for multiclass classificationCombined with cross-entropy: ∂L/∂z_k = ŷ_k - y_k (clean gradient)

2.9 Deep Learning in Action: Multiclass Classification and Regression

Two fundamental problem types in supervised deep learning are multiclass classification (predicting one of K discrete categories) and regression (predicting a continuous value).

2.9.1 Multiclass Classification Pipeline

- Architecture: Input → hidden layers (Dense/Conv) → Dense(K) → Softmax
- Loss: Categorical Cross-Entropy (sparse for integer labels)
- Metric: Accuracy = correct predictions / total
- Evaluation: Confusion matrix, precision/recall/F1 per class, ROC-AUC
- Example: Classifying handwritten digits (MNIST: 10 classes), ImageNet (1000 classes)

MNIST Example Architecture

Input: (batch, 28, 28, 1) → Flatten → (batch, 784)Dense(128) + ReLU → Dense(64) + ReLU → Dense(10) + SoftmaxLoss: sparse_categorical_crossentropyOptimizer: Adam(lr=0.001)Typical accuracy: ~98% on test set after 10 epochs

2.9.2 Regression Pipeline

- Architecture: Input → hidden layers → Dense(1) (no activation on output)
- Loss: MSE or MAE (or Huber for robustness)
- Metric: RMSE, MAE, R^2 score
- Normalisation: Always normalise input features (zero mean, unit variance)
- Example: Predicting house prices, stock values, weather, medical measurements

2.9.3 Application Examples

Deep learning has powered breakthrough applications across domains:

- Image Classification: ResNet/EfficientNet achieve >90% top-5 accuracy on ImageNet
- Object Detection: YOLO (You Only Look Once) — real-time detection in video streams
- NLP Classification: BERT fine-tuned on sentiment analysis, toxicity detection, NLI
- Speech Recognition: End-to-end CTC networks / Transformer ASR (Whisper)
- Medical Imaging: DL models match radiologist performance on skin cancer, diabetic retinopathy

2.10 Overfitting and Underfitting

Generalisation — the ability to perform well on unseen data — is the ultimate goal of machine learning. Overfitting and underfitting represent the two ways this can fail.

2.10.1 Underfitting (High Bias)

Underfitting occurs when a model is too simple to capture the underlying patterns. High training loss AND high validation loss. Caused by: insufficient model capacity, too few training epochs, too much regularisation, poorly engineered features.

- Solutions: Increase model capacity (more layers/neurons), train longer, reduce regularisation, add relevant features

2.10.2 Overfitting (High Variance)

Overfitting occurs when a model memorises the training data including noise. Very low training loss but high validation loss (generalisation gap). Caused by: excessive model capacity, insufficient training data, insufficient regularisation.

Detecting Overfitting

Training loss: continuously decreasingValidation loss: decreases, then starts INCREASING → overfitting has begunGeneralisation gap = val_loss - train_loss (should be small)Monitor both curves every epoch — use early stopping to halt at minimum val_loss

2.10.3 Regularisation Techniques to Combat Overfitting

- L1/L2 Weight Regularisation: Penalise large weights in the loss function
- Dropout: Randomly zero neurons — prevents co-adaptation, acts as ensemble
- Batch Normalisation: Normalises activations — mild regularisation effect
- Data Augmentation: Expand training set with transformations (flip, crop, rotate, colour jitter)
- Early Stopping: Stop training when validation loss stops improving
- Reduce Model Complexity: Fewer layers/neurons, shallower trees
- Cross-Validation: Detect overfitting with held-out validation set
- Transfer Learning: Pretrained models have learned robust generalisable features

2.10.4 The Bias-Variance-Complexity Trade-off

As model complexity increases: training error decreases monotonically. Validation error forms a U-shape — decreases initially (bias reduces), then increases (variance increases). The optimal model

sits at the bottom of the U. Modern deep learning with large datasets can operate in the 'double descent' regime where very large models continue to improve beyond this classical optimum.

QUESTION & ANSWER SECTION

50 questions organised by marks. Comprehensive answers suitable for university examinations.

— SECTION A: 10-MARK QUESTIONS —

Q1. Differentiate Deep Learning from Machine Learning. Discuss the evolution of AI through historical epochs and explain the three key drivers of the deep learning revolution. [10 Marks]

Answer:

Machine Learning (ML) is a subset of AI that learns patterns from labeled data using algorithms like SVMs, decision trees, and linear models. It typically requires manual feature engineering — domain experts design input representations. Performance plateaus with more data. Deep Learning (DL) is a subset of ML using multi-layered artificial neural networks (ANNs). It automatically learns hierarchical feature representations from raw data. Performance scales with data and compute.Key Differences: ML needs manual features; DL learns them end-to-end. ML works with small data; DL needs large datasets. ML is interpretable; DL is a 'black box.' ML uses CPUs; DL demands GPUs/TPUs.Historical Epochs: (1) Symbolic AI (1950s-80s): Rule-based systems, expert systems, Turing Test (1950), Perceptron (1958). (2) First & Second AI Winters (1974-80, 1987-93): Minsky & Papert (1969) showed perceptron limitations; funding cuts. (3) Statistical ML (1990s-2000s): SVMs, boosting, probabilistic models dominate. (4) DL Renaissance (2006+): Hinton's DBNs (2006); AlexNet wins ImageNet 2012 by 10%+; GANs (2014); Transformers (2017); GPT-4, LLaMA (2023+).Three Drivers: (1) DATA — internet, smartphones, sensors generate billions of labeled examples. (2) COMPUTE — GPUs (thousands of CUDA cores), TPUs (100+ TFLOPS), cloud computing. (3) ALGORITHMS — ResNets, Transformers, Adam optimizer, Dropout, BatchNorm, open frameworks (PyTorch, TensorFlow). Together these caused a phase transition: deep neural networks became practical and dominant.

Q2. Explain the hierarchy of tensors: scalars, vectors, matrices, and tensors. Describe how each is used in deep learning with mathematical notation, real-world data examples, and tensor operations. [10 Marks]

Answer:

Scalars (Rank-0 tensors): Single numbers. $x \in \mathbb{R}$. Examples: learning rate $\eta=0.001$, loss=0.47. No shape — shape=().Vectors (Rank-1): Ordered 1D arrays. $x \in \mathbb{R}^n$. Examples: feature vector of one example (784,), word embedding (300,), bias vector. Operations: dot product $x \cdot y=\Sigma x_i y_i$, L2 norm $||x||=\sqrt{(\Sigma x_i^2)}$, scalar multiplication αx.Matrices (Rank-2): 2D rectangular arrays. $A \in \mathbb{R}$^(m×n). Examples: weight matrix W connecting layers, dataset of n examples × p features. Operations: matrix multiply C=AB (m×k)×(k×n)→(m×n); transpose A^T; element-wise $A \odot B$. Key: a linear layer computes output = W*input + b using matrix-vector multiplication.Tensors (Rank ≥ 3): Rank-3: (N,H,W) grayscale image batches or (N,T,d) NLP sequences. Rank-4: (N,H,W,C) colour image batches — the standard CNN input. Rank-5: (N,T,H,W,C) video batches.Core Operations: Broadcasting extends lower-rank tensors to match higher-rank (bias (d,) + activations (B,d)). Reshape/flatten: (B,28,28,1)→(B,784) for dense layers. Reduction: mean over batch dimension. Element-wise: ReLU(x)=max(0,x) applied to each element. Contraction/einsum: generalised

matrix operations. In PyTorch: torch.tensor, .shape, .reshape, .transpose, .matmul (@). GPU acceleration: tensor.to('cuda') — element-wise ops parallelised across thousands of cores.

Q3. Explain backpropagation in detail. Derive the weight update equations for a two-layer neural network using the chain rule. Discuss computational graphs and automatic differentiation. [10 Marks]

Answer:

Backpropagation computes the gradient of the loss w.r.t. every parameter by applying the chain rule backwards through the network.Setup — Two-layer network: Forward pass: z1=W1*x+b1; a1=σ(z1); z2=W2*a1+b2; ŷ=softmax(z2); L=CE(ŷ, y).Chain Rule: ∂L/∂W1 = ∂L/∂z2 * ∂z2/∂a1 * ∂a1/∂z1 * ∂z1/∂W1.Layer 2 gradients: δ2 = ∂L/∂z2 = ŷ - y (for softmax+CE: clean gradient). ∂L/∂W2 = δ2 * $a1^T$; ∂L/∂b2 = δ2.Layer 1 gradients: δ1 = ($W2^T$ * δ2) ⊙ σ'(z1). σ'(z1): derivative of activation at z1 — e.g., for ReLU: 1 if z1>0 else 0. ∂L/∂W1 = δ1 * x^T; ∂L/∂b1 = δ1.General pattern: δ_l = (W_{l+1}^T * δ_{l+1}) ⊙ σ'(z_l). ∂L/∂W_l = δ_l * a_{l-1}^T.Computational Graphs: Operations are nodes; tensors are edges in a DAG. Forward pass: computes outputs, stores intermediate values. Backward pass: traverses DAG in reverse, multiplying local Jacobians. Autograd (PyTorch): builds graph dynamically during forward. loss.backward() triggers reverse traversal. Each parameter's .grad stores gradient. Advantages: works for any differentiable computation — no manual derivation needed. Importance of storing activations: ∂L/∂W_l requires a_{l-1} — hence memory usage = O(depth × batch_size × layer_size) during training.

Q4. Describe gradient descent variants in detail (Batch GD, SGD, Mini-batch SGD). Explain learning rate effects and common learning rate schedules. Compare SGD with Momentum, RMSProp, and Adam. [10 Marks]

Answer:

Batch GD: θ ← θ - η*(1/n)*Σ∇L(θ;xi,yi). True gradient — smooth convergence. Prohibitively slow for large n; cannot update until all n examples processed.SGD: Uses one example per step. θ ← θ - η*∇L(θ;xi,yi). Fast updates, can escape local minima, online learning possible. High variance — loss oscillates. In practice rarely used in pure form.Mini-batch SGD: Uses batch B (32-512). Balances GPU efficiency (vectorised ops) with gradient noise. Industry standard for deep learning. Larger batch → smoother gradients, risk of sharp minima (worse generalisation). Smaller batch → more noise, better regularisation effect.Learning Rate: Too large → divergence (overshoots minimum). Too small → slow convergence, stuck in local minima. Schedules: Step decay (×γ every k epochs); Cosine annealing (smooth oscillation); Warm-up (start small, increase then decay); Cyclical LR (oscillate — helps escape saddle points).SGD + Momentum: v_t=β*v_{t-1}+(1-β)*g_t; θ←θ-η*v_t. Dampens oscillations, accelerates in consistent direction. β=0.9 standard.RMSProp: s_t=ρ*s_{t-1}+(1-ρ)*g_t^2; θ←θ-(η/√(s_t+ε))*g_t. Adapts LR per parameter. Good for RNNs and non-stationary.Adam: Combines momentum (m_t=β1*m_{t-1}+(1-β1)*g_t) and adaptive LR (v_t=β2*v_{t-1}+(1-β2)*g_t^2) with bias correction. θ←θ-η*m̂_t/(√v̂_t+ε). Default: η=0.001, β1=0.9, β2=0.999. Works well out of the box — the standard choice. AdamW adds decoupled weight decay — standard for Transformers.

Q5. Explain all major activation functions used in deep learning (Sigmoid, Tanh, ReLU, Leaky ReLU, GELU, Softmax). Discuss vanishing gradients and how modern activations solve this problem. [10 Marks]

Answer:

Activation functions introduce non-linearity — without them, deep networks reduce to one linear transformation.Sigmoid: $\sigma(x)=1/(1+e^{-x})$, output (0,1). Max derivative 0.25. Saturates for $|x|>3$ → vanishing gradients. Not zero-centered → slow convergence. ONLY for binary classification output.Tanh: $\tanh(x)=(e^x-e^{-x})/(e^x+e^{-x})$, output (-1,1). Zero-centered (better). Still saturates. Used in LSTM/GRU gates and some RNN hidden states.Vanishing Gradient Problem: In deep networks with sigmoid/tanh, backprop multiplies local gradients (<0.25) repeatedly. After L layers, gradient $\propto 0.25^L$ → vanishes exponentially. Lower layers learn almost nothing.ReLU: $f(x)=\max(0,x)$. Derivative: 1 for $x>0$, 0 for $x<0$. Non-saturating for positive inputs → fast convergence. Dying ReLU problem: neurons stuck at $x<0$ stop learning permanently. Standard for CNNs and MLPs.Leaky ReLU: $f(x)=x$ if $x>0$ else αx ($\alpha\approx0.01$). Non-zero gradient for $x<0$ → fixes dying ReLU. PReLU: α learnable.ELU: Smooth negative part, zero-centered outputs. SELU: Self-normalising — maintains unit mean/variance. Good for very deep networks without BatchNorm.GELU: $x*\Phi(x)$ where Φ is the Gaussian CDF. Smooth everywhere, stochastic interpretation, best empirical performance on NLP tasks. Standard in BERT, GPT, modern Transformers. SiLU/Swish: x*sigmoid(x) — similar.Softmax: $\text{softmax}(z)_k=e^{z_k}/\Sigma e^{z_j}$. Converts logits to probabilities summing to 1. Multiclass output layer. Combined with cross-entropy: $\partial L/\partial z_k = \hat{y}_k - y_k$.

Q6. Explain loss functions for both classification and regression. Derive the gradient of categorical cross-entropy with softmax and discuss how the choice of loss function affects model training. [10 Marks]

Answer:

The loss function defines the training objective — it quantifies how wrong the model is. Choosing the wrong loss can cause slow convergence, instability, or a mismatch between training objective and business metric.Regression Losses: $MSE=(1/n)\Sigma(y_i-\hat{y}_i)^2$. Squares errors → outliers dominate. Gradient: $-2(y_i-\hat{y}_i)/n$. Smooth, differentiable. Best when errors are normally distributed. $MAE=(1/n)\Sigma|y_i-\hat{y}_i|$. Linear penalty → outlier robust. Non-differentiable at 0 (use subgradient). Huber: MSE for $|e|<\delta$, MAE outside → best of both.Binary Classification: $BCE=-(1/n)\Sigma[y*\log(\hat{y})+(1-y)*\log(1-\hat{y})]$. Requires sigmoid output $\hat{y}\in(0,1)$. Derived from maximum likelihood with Bernoulli distribution.Multiclass: $CCE=-(1/n)\Sigma_i \Sigma_k y_{ik}*\log(\hat{y}_{ik})$. Requires softmax output. For K classes with one-hot y: $CCE=-(1/n)\Sigma \log(\hat{y}_{i,correct})$.Derivation of Softmax+CCE Gradient: $\hat{y}_k=e^{z_k}/\Sigma e^{z_j}$. $L=-\Sigma_k y_k*\log(\hat{y}_k)$. $\partial L/\partial z_k$: Using chain rule through softmax: $\partial L/\partial z_k = \Sigma_j (\partial L/\partial \hat{y}_j)*(\partial \hat{y}_j/\partial z_k)$. After algebra: $\partial L/\partial z_k = \hat{y}_k - y_k$. This is the clean, numerically stable gradient — the softmax output minus the one-hot target. This gradient is: zero where prediction matches truth, positive where model over-predicts, negative where it under-predicts.Effect on Training: Loss shape determines gradient landscape. MSE creates quadratic bowl (smooth). Cross-entropy has steeper gradients when predictions are wrong → faster learning signal. Focal Loss: down-weights easy examples → better for class imbalance. Contrastive/triplet loss: metric learning for embeddings.

Q7. Describe overfitting and underfitting in deep learning. Explain at least six regularisation techniques with mathematical formulations and discuss the double descent phenomenon. [10 Marks]

Answer:

Underfitting (High Bias): Model too simple to capture data patterns. Both training and validation loss are high. Causes: insufficient capacity, too few epochs, over-regularisation. Solution: larger model, more epochs, less regularisation.Overfitting (High Variance): Model memorises training

noise. Low train loss, high val loss. Generalisation gap = val_loss - train_loss. Causes: model too complex relative to data size.Regularisation Techniques:1. L2 (Weight Decay): Add $(\lambda/2)||W||^2$ to loss. Gradient: ∇L += λW. Penalises large weights → smoother decision boundaries. Bayesian interpretation: Gaussian prior on weights.2. L1 (Lasso): Add $\lambda||W||_1$. Promotes sparsity → implicit feature selection. Gradient: λ*sign(W).3. Dropout: During training, zero activations with probability p. Gradient flows only through active neurons. At inference: scale by (1-p). Equivalent to ensemble of 2^n models. Typical p=0.3-0.5 for FC layers.4. Batch Normalisation: Normalises activations per mini-batch → reduces internal covariate shift. Acts as mild regulariser (batch statistics add noise). Enables higher learning rates.5. Data Augmentation: Artificially expand training set: random flips, crops, rotations, colour jitter (images); synonym replacement, back-translation (NLP); speed/pitch perturbation (audio). Trains model on more diverse examples.6. Early Stopping: Monitor validation loss; stop when it stops improving; restore best checkpoint. Implicit regularisation — stops before model overfits.7. Label Smoothing: Replace hard one-hot labels with soft: y_smoothed = (1-ε)*y + ε/K. Prevents overconfidence, improves calibration.Double Descent: Classical wisdom: test error follows U-curve vs. model complexity. Modern finding (Belkin et al., 2019): for very large models (overparameterised regime, parameters > training examples), test error can decrease again — second descent. Interpolating models that exactly fit training data can still generalise well if they are implicitly biased toward simple solutions (minimum norm). Relevant for large Transformers and foundation models.

Q8. Explain the different types of layers in a deep neural network. Describe Dense, Convolutional, Pooling, Recurrent, Embedding, and Normalisation layers with their mathematical formulations. [10 Marks]

Answer:

Layers are the atoms of deep learning. Each transforms an input tensor to an output tensor via learned parameters.1. Dense (Fully Connected): output = σ(W*input + b). W ∈ ℝ^(out×in), b ∈ ℝ^out. Every input connected to every output — O(in×out) parameters. Captures global relationships. Computationally expensive for high-dim inputs. Used in: classification heads, MLPs, Transformer FFN sublayers.2. Convolutional Layer: Applies F filters of size k×k across spatial locations. Output[i,j,f] = Σ_m Σ_n Σ_c Input[i*s+m, j*s+n, c] * Filter[m,n,c,f] + b_f. Weight sharing (same filter at all positions) → translation equivariance. Parameters: k*k*C_in*C_out + C_out. Output size: floor((W-k+2P)/S)+1. Hierarchical feature learning: edges → textures → objects.3. Pooling: Max pooling: output=max over k×k window. Average pooling: output=mean. Global Average Pooling: reduces (H,W,C) → (C) — used before classification head in modern CNNs. No learnable parameters.4. Recurrent (RNN/LSTM/GRU): RNN: h_t = tanh(W_h*h_{t-1} + W_x*x_t + b). Processes sequences with shared parameters. LSTM: adds cell state C_t and forget/input/output gates — handles long-range dependencies (C_t = f_t⊙C_{t-1} + i_t⊙C̃_t). GRU: simplified version with reset and update gates.5. Embedding Layer: Lookup table E ∈ ℝ^(V×d). token_id → E[token_id] ∈ ℝ^d. Learnable dense representations of discrete tokens. Used in NLP, recommendation systems, any categorical feature. No activation — pure lookup.6. Batch Normalisation: For each mini-batch B: x̂=(x-μ_B)/√(σ²_B+ε); output=γ*x̂+β. γ,β learnable per-feature. At inference: uses running statistics computed during training. Advantages: higher LR, less sensitive to init, mild regularisation.7. Layer Normalisation: Normalises over feature dimension (not batch). Better for Transformers/RNNs where batch statistics are unreliable. output = γ*(x-μ)/σ + β, computed per sample.

Q9. Describe the training pipeline for a deep learning model end-to-end: data preprocessing, model design, loss function choice, optimiser selection, training loop, evaluation, and deployment considerations. [10 Marks]

Answer:

End-to-End Deep Learning Pipeline:1. Data Collection & Preprocessing: Gather raw data (images, text, tabular). Clean: remove duplicates, handle missing values, fix labels. Split: train (70-80%), validation (10-15%), test (10-15%) — never tune on test set. Normalise: images to [0,1] or zero-mean/unit-variance; text: tokenise, build vocabulary; tabular: standardise features.2. Data Loading: DataLoaders with batching, shuffling, multi-worker prefetching. Data Augmentation applied on-the-fly during training only.3. Model Design: Choose architecture matching data type (CNN for images, Transformer for text, MLP for tabular). Define depth (number of layers), width (neurons/filters per layer), activation functions, regularisation (dropout rate, weight decay). Transfer learning from pretrained model when data is limited.4. Loss Function: Binary CE for binary classification; Categorical CE for multiclass; MSE/Huber for regression. Ensure loss matches output activation (sigmoid → BCE; softmax → CCE; linear → MSE).5. Optimiser: Adam (lr=0.001) as default. SGD with momentum for CNNs (often better final accuracy with tuning). AdamW + cosine annealing for Transformers.6. Training Loop: zero_grad() → forward pass → compute loss → loss.backward() → gradient clipping → optimizer.step(). Repeat for all mini-batches (1 epoch). Monitor train & val loss/metric every epoch. Early stopping based on val loss. Checkpoint best model.7. Evaluation: Report metrics on held-out test set (never used during training/selection). Classification: accuracy, F1, ROC-AUC, confusion matrix. Regression: RMSE, MAE, R^2.8. Deployment: Export model (ONNX, TorchScript, TensorFlow SavedModel). Optimise inference: quantisation (INT8), pruning, distillation. Serve via REST API (FastAPI, TensorFlow Serving). Monitor: data drift, model degradation, latency.

Q10. Explain multiclass classification and regression using deep learning. Discuss network architectures, loss functions, evaluation metrics, and provide worked examples for each. [10 Marks]

Answer:

Multiclass Classification — Predict one of K discrete categories:Architecture: Input → [Feature Extractor: Dense/Conv/Transformer layers] → Dense(K, no activation) → Softmax → K probabilities. Final Dense layer has K units = number of classes.Loss: Categorical Cross-Entropy: $L = -(1/n)\Sigma \log(\hat{y}_{i,c_i})$, where c_i is the correct class. Sparse CCE for integer labels (no one-hot needed).Evaluation Metrics: Accuracy = TP_all/n (misleading if imbalanced). Precision_k = TP_k/(TP_k+FP_k). Recall_k = TP_k/(TP_k+FN_k). F1_k = 2*P*R/(P+R). Macro/Micro averaging for multi-class. Confusion matrix C where C[i,j] = examples of class i predicted as class j.Worked Example — MNIST Digit Classification: Input: (28,28,1) images. Architecture: Flatten → Dense(256, ReLU) → Dropout(0.3) → Dense(128, ReLU) → Dropout(0.3) → Dense(10, Softmax). Loss: sparse_categorical_crossentropy. Optimizer: Adam(0.001). Result: ~98.5% test accuracy in 10 epochs.Regression — Predict a continuous value:Architecture: Input → [Feature Extractor] → Dense(1, linear activation) — NO softmax, no sigmoid on output layer. Output range is unconstrained unless you add sigmoid (0,1) or ReLU (≥0).Loss: MSE = $(1/n)\Sigma(y_i-\hat{y}_i)^2$ (standard). Huber for outlier robustness. MAE if median preferred.Evaluation: RMSE = $\sqrt{MSE}$. MAE = mean absolute error. $R^2 = 1 - SS_{res}/SS_{tot}$ (1=perfect, 0=baseline mean predictor).Worked Example — Boston Housing: Input: 13 features, normalised. Architecture: Dense(64, ReLU) → Dense(32, ReLU) → Dense(1). Loss: MSE. Key: normalise all inputs (zero mean, unit variance) — critical for convergence. Monitor val_MAE. Feature importance via permutation. Typical R^2: 0.85-0.92.

Q11. Explain how deep learning models are visualised and interpreted. Discuss methods for visualising features, gradients, and activations, and address the interpretability challenge in deep learning. [10 Marks]

Answer:

Visualisation and interpretability are critical for debugging, trusting, and deploying deep learning models responsibly.1. Training Dynamics Visualisation (TensorBoard/WandB): Loss curves (train vs. val) — detect overfitting, underfitting, divergence. Accuracy over epochs. Learning rate schedule. Gradient norms per layer — detect vanishing/exploding gradients. Weight histograms — check if weights are well-distributed or saturated.2. Feature Map Visualisation (CNNs): After each convolutional layer, display the activation maps for a given input image. Early layers (conv1): detect edges, colours, textures — Gabor-like filters. Middle layers: detect shapes, object parts. Final layers: high-level semantic features (faces, wheels, eyes). Reveals what patterns each filter is 'looking for.'3. Filter Visualisation: Optimise input to maximally activate a specific filter: $x^* = \text{argmax}_x \text{activation_filter}(x)$. Produces abstract patterns — confirms filters learn meaningful visual concepts.4. Grad-CAM (Gradient-weighted Class Activation Mapping): Computes importance of each spatial location for a specific class prediction: $\alpha_k = (1/Z)\Sigma\Sigma(\partial y^c/\partial A^k_{ij})$ — global average pooling of gradients. Heatmap = $\text{ReLU}(\Sigma\ \alpha_k * A^k)$ — highlights discriminative regions. Does not require model modification. Useful for medical imaging (shows which tumour region caused the prediction).5. t-SNE / UMAP of Embeddings: Extract penultimate layer representations for all test examples. Reduce to 2D with t-SNE or UMAP. Good representations: same-class clusters clearly separated — confirms the network has learned discriminative features.6. LIME (Local Interpretable Model-agnostic Explanations): Perturb input locally, fit a simple interpretable model (linear) to approximate neural network behaviour in that neighbourhood. Explains individual predictions.7. SHAP (SHapley Additive exPlanations): Assigns each feature an importance value based on Shapley values from game theory. Consistent, theoretically principled. Works model-agnostically.8. Attention Visualisation: In Transformers, visualise attention weights as heatmaps showing which tokens attend to which — reveals grammatical, semantic, coreference patterns. Limitation: attention ≠ explanation (high attention does not necessarily imply causal importance).

Q12. Describe the role of optimisers in training deep learning models. Compare SGD, Adam, AdaGrad, and RMSProp. Explain the effect of hyperparameters and discuss techniques like gradient clipping and learning rate warm-up. **[10 Marks]**

Answer:

Optimisers update model parameters using gradients to minimise the loss. They differ in how they accumulate gradient history and adapt learning rates per parameter.SGD (Vanilla): $\theta \leftarrow \theta - \eta * g_t$. Simple but requires careful LR tuning. Uniform step size for all parameters — inefficient. With Momentum: $v_t=\beta*v_{t-1}+(1-\beta)*g_t$; $\theta\leftarrow\theta-\eta*v_t$. Accumulates velocity → faster in consistent gradient directions, damped oscillations. $\beta=0.9$ standard. Nesterov: uses gradient at lookahead position for better convergence.AdaGrad: $G_t \mathrel{+}= g_t^2$; $\theta\leftarrow\theta-(\eta/\sqrt{(G_t+\varepsilon)})*g_t$. Per-parameter adaptive LR. Rare features get larger updates — great for sparse NLP features. Problem: G_t grows monotonically → LR shrinks to 0 → learning stops.RMSProp: Fixes AdaGrad: $s_t=\rho*s_{t-1}+(1-\rho)*g_t^2$ (EMA). $\theta\leftarrow\theta-(\eta/\sqrt{(s_t+\varepsilon)})*g_t$. Non-monotonic: LR can increase for parameters not recently updated. $\rho=0.9$ typical. Good for RNNs.Adam: $m_t=\beta1*m_{t-1}+(1-\beta1)*g_t$; $v_t=\beta2*v_{t-1}+(1-\beta2)*g_t^2$. Bias correction: $\hat{m}=m/(1-\beta1^t)$, $\hat{v}=v/(1-\beta2^t)$. $\theta\leftarrow\theta-\eta*\hat{m}/(\sqrt{\hat{v}}+\varepsilon)$. Combines momentum and adaptive LR. $\beta1=0.9$, $\beta2=0.999$, $\eta=0.001$. AdamW: decoupled weight decay (adds $-\lambda\theta$ directly, not via gradient) — better for Transformers. Lion: uses sign of Adam's momentum update — memory efficient.Gradient Clipping: Clips gradient norm to threshold τ: if $||g||>\tau$, $g\leftarrow\tau*g/||g||$. Prevents exploding gradients — essential for RNNs and very deep networks. Typically $\tau=1.0$ or 5.0.Learning Rate Warm-up: Start with $\eta=0$, linearly increase to η_{max} over first N_{warmup} steps, then decay (cosine/step). Prevents instability in early training when model is far from optimum. Standard practice for Transformers (Vaswani et al., 2017) and BERT fine-tuning.

Q13. What is the 'Elephant in the Room' in deep learning? Discuss the interpretability challenge, adversarial examples, data bias, and current approaches to making deep learning safer and more trustworthy. [10 Marks]

Answer:

The 'elephant in the room' metaphor refers to the critical interpretability challenge that is often ignored in favour of accuracy benchmarks: deep learning models with millions of parameters and non-linear interactions are fundamentally opaque — we cannot easily explain specific predictions.The Interpretability Challenge: A ResNet-50 with 25M parameters transforms an image through 50 layers of non-linear computations. No human-comprehensible rule can summarise this process. Yet these models make decisions in healthcare (cancer diagnosis), finance (loan approval), criminal justice (recidivism prediction) — where explanations are legally and ethically mandatory.Adversarial Examples (Szegedy et al., 2014): Adding tiny, human-imperceptible perturbations (ε-bounded noise) to an image can cause a 99% confident wrong prediction — a panda classified as a gibbon. FGSM attack: x_adv = x + ε*sign(∇_x L). Deep models learn non-robust features that do not correspond to human-meaningful patterns. Adversarial training (training on adversarial examples) improves robustness but reduces accuracy.Dataset Bias: Models learn and amplify biases in training data. A face recognition model trained on non-diverse data performs poorly on underrepresented groups. Language models learn gender, racial, and cultural stereotypes from web text. Mitigation: diverse training data, fairness constraints (equalised odds, demographic parity), post-processing calibration.Hallucination (LLMs): Language models generate factually incorrect but plausible-sounding text. Caused by learning statistical patterns rather than grounded factual knowledge. Retrieval-augmented generation (RAG) and RLHF mitigate but do not eliminate.Current Approaches: LIME and SHAP: post-hoc local explanations. Grad-CAM: gradient-based heatmaps for CNNs. Mechanistic Interpretability (Anthropic): understand circuits — specific computation patterns within model. Concept Activation Vectors (TCAV): test what concepts a model responds to. Formal Verification: prove bounds on model behaviour (limited to small models). Certified Robustness: guarantee model output within ε-ball around input. Regulatory Response: EU AI Act requires high-risk AI systems to provide explanations.

Q14. Explain real-world data representation using tensors. How are images, text, audio, and tabular data represented? Describe preprocessing pipelines and the importance of data normalisation in deep learning. [10 Marks]

Answer:

Real-world data representation as tensors is the first step in any deep learning pipeline. Incorrect representation or poor preprocessing can prevent convergence regardless of model quality.Images: Grayscale: (H, W) rank-2 tensor, values 0-255. Colour (RGB): (H, W, 3) rank-3, 3 channels. Batch: (N, H, W, 3) rank-4 — PyTorch uses (N, 3, H, W) channels-first. Preprocessing: resize to fixed dimensions (e.g., 224×224 for ImageNet models), normalise to [0,1] or per-channel z-score: (x - μ)/σ with ImageNet stats μ=[0.485,0.456,0.406], σ=[0.229,0.224,0.225]. Augmentation: random flip, crop, rotation, colour jitter.Text: Tokenise into subwords (BPE, WordPiece — e.g., 'unhappy' → ['un','happy']). Build vocabulary (30k-100k tokens). Map to integers: [2, 847, 93, ...]. Sentence of length T: (T,) rank-1. Batch B sentences padded to same length: (B, T). Embed with embedding matrix E (V×d): (B, T, d) rank-3. Add position encodings. Normalise: not needed for token IDs; embeddings initialised N(0,1).Audio: 1-second at 16kHz: (16000,) rank-1. Mel spectrogram: short-time Fourier transform → (time_frames, mel_bins) rank-2 — treat as grayscale image. Batch: (N, time_frames, mel_bins). Normalise: subtract mean, divide by std across dataset.Tabular Data: N examples × p features: (N, p) rank-2. Numerical features: z-score normalisation (μ=0, σ=1) or min-max to [0,1]. Categorical features: one-hot encoding (high cardinality) or learnable embeddings (better for ≥10 categories). Missing values: impute (mean, median) or train separate missing indicator.Importance of Normalisation:

Un-normalised features cause: (1) Gradient descent steps to be dominated by large-scale features — others learn slowly. (2) Activation saturation (sigmoid/tanh) → vanishing gradients. (3) Very large initial weights → unstable training. Normalisation puts all features on equal footing and keeps activations in the linear regime of saturating activation functions.

— SECTION B: 5-MARK QUESTIONS —

Q16. What is a tensor? Explain ranks with examples. [5 Marks]

Answer:

A tensor is a generalisation of scalars, vectors, and matrices to arbitrary dimensions. Rank 0 (scalar): single number — e.g., loss = 0.42. Rank 1 (vector): 1D array — e.g., feature vector (784,). Rank 2 (matrix): 2D grid — e.g., weight matrix (128, 64), dataset (n, p). Rank 3: e.g., batch of greyscale images (32, 28, 28). Rank 4: batch of RGB images (32, 224, 224, 3). Rank 5: video batch (8, 16, 112, 112, 3). Tensors are implemented in NumPy (ndarray) and PyTorch (torch.Tensor). Key properties: dtype (float32, int64), shape, device (CPU/GPU). Operations: element-wise arithmetic, matmul, reshape, slice, reduce (sum, mean).

Q17. Explain the chain rule and how it applies to backpropagation. [5 Marks]

Answer:

Chain rule (calculus): For composite function f(g(x)), df/dx = (df/dg)*(dg/dx). In deep networks, the loss L is a composition of many functions: L = fN(fN-1(...f1(x))). Backpropagation applies the chain rule recursively: $\partial L/\partial W_l = \partial L/\partial a_N * \partial a_N/\partial a_{N-1} * \ldots * \partial a_{l+1}/\partial a_l * \partial a_l/\partial W_l$. In practice: compute $\delta_L = \partial L/\partial z_L$ (output layer error). Propagate: $\delta_l = (W_{l+1}^T * \delta_{l+1}) \odot \sigma'(z_l)$. Weight gradients: $\partial L/\partial W_l = \delta_l * a_{l-1}^T$. All intermediate activations stored during forward pass for use in backward pass. Autograd does this automatically by building a computational graph.

Q18. Differentiate between batch gradient descent, SGD, and mini-batch gradient descent. [5 Marks]

Answer:

Batch GD: Computes gradient over entire training set per update. True gradient → smooth convergence. Too slow for large datasets (single update per epoch). SGD (Stochastic): Gradient from 1 random example per update. Very fast iterations; noisy gradient → high variance; can escape local minima; online learning capable. Mini-batch (standard): Gradient from B examples (32-512). Vectorised computation (GPU efficient). Balanced noise vs. accuracy. Most practical choice. Comparison: Speed: SGD > Mini-batch > Batch. Stability: Batch > Mini-batch > SGD. Memory: Batch requires full dataset; Mini-batch requires B examples. In deep learning, 'SGD' typically refers to mini-batch SGD.

Q19. What is the vanishing gradient problem? How do ReLU and residual connections address it? [5 Marks]

Answer:

Vanishing gradients: in backpropagation, gradients are multiplied by activation derivatives at each layer. Sigmoid derivative max = 0.25; after L layers, gradient ∝ 0.25^L → approaches 0 exponentially. Lower layers receive near-zero gradients — cannot learn. ReLU Fix: f(x)=max(0,x). Derivative = 1 for x>0 — no shrinkage for positive inputs. Gradients pass through relu layers unchanged (for active neurons). Residual Connections (ResNets): Output = F(x) + x. Gradient of loss: $\partial L/\partial x = \partial L/\partial output * (\partial F/\partial x + 1)$. The '+1' ensures a direct gradient path to every

layer regardless of F's gradient. 100-layer network can train effectively. Batch Normalisation also helps by keeping activations in the linear regime of activation functions.

Q20. Explain dropout regularisation. Why does it work? [5 Marks]

Answer:

Dropout (Srivastava et al., 2014): During training, each neuron independently set to 0 with probability p (dropout rate, typically 0.2-0.5). Inverted dropout scales active neurons by 1/(1-p) to maintain expected activation magnitude. At inference: no dropout — all neurons active. Why it works: (1) Prevents co-adaptation: neurons cannot rely on specific other neurons — must learn robust, independent features. (2) Ensemble interpretation: at each training step, a different sub-network is trained. With n dropout neurons, there are 2^n possible networks. Final network approximates their average. (3) Noise injection: stochastic zeroing acts as input noise, similar to data augmentation. SpatialDropout: drops entire feature maps (better for CNNs). DropConnect: drops weights instead of neurons (more general). BatchNorm often reduces need for Dropout — can conflict.

Q21. What is Adam optimiser? How does it differ from SGD? [5 Marks]

Answer:

Adam (Adaptive Moment Estimation, Kingma & Ba, 2015): Maintains two moment estimates per parameter. First moment (momentum): $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ ($\beta_1=0.9$) — smoothed gradient direction. Second moment (adaptive LR): $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ ($\beta_2=0.999$) — smoothed squared gradient magnitude. Bias correction: $\hat{m}=m/(1-\beta_1^t)$, $\hat{v}=v/(1-\beta_2^t)$. Update: $\theta \leftarrow \theta - \eta \hat{m}/(\sqrt{\hat{v}}+\varepsilon)$. Effect: parameters with large recent gradients get small LR ($\sqrt{\hat{v}}$ is large); parameters with small gradients get large LR. SGD uses same η for all parameters; Adam adapts per parameter. SGD needs careful LR tuning; Adam works well with defaults (lr=0.001). Adam converges faster; SGD with momentum sometimes finds better final solutions (flatter minima). AdamW adds proper L2 weight decay (decoupled from adaptive LR).

Q22. Describe the softmax function and categorical cross-entropy loss. [5 Marks]

Answer:

Softmax: Converts raw logits $z \in \mathbb{R}^K$ to probability distribution over K classes. $\text{softmax}(z)_k = e^{z_k} / \sum_j e^{z_j}$. All outputs $\in (0,1)$, sum to 1. Numerically stable: subtract max(z) before exponentiating to prevent overflow. Used as final layer for multiclass classification. Categorical Cross-Entropy: $L = -(1/n)\sum_i \sum_k y_{ik} * \log(\hat{y}_{ik})$. For one-hot y: $L = -(1/n)\sum_i \log(\hat{y}_{i,c_i})$ — just the log-probability of the correct class. Maximum likelihood interpretation: minimising CE = maximising log-likelihood under categorical distribution. Gradient of softmax+CE: $\partial L/\partial z_k = \hat{y}_k - y_k$ — predicted probability minus target. This is clean, numerically stable. If model predicts 0.9 for correct class: gradient≈-0.1. If 0.1: gradient≈-0.9 — large signal.

Q23. What is batch normalisation and why is it important? [5 Marks]

Answer:

Batch Normalisation (Ioffe & Szegedy, 2015): Normalises activations per feature across a mini-batch. For feature j in mini-batch B of size m: $\mu_j = (1/m)\Sigma x_{ij}$; $\sigma^2_j = (1/m)\Sigma(x_{ij}-\mu_j)^2$; $\hat{x}_{ij} = (x_{ij}-\mu_j)/\sqrt{(\sigma^2_j+\varepsilon)}$. Output: $y_{ij} = \gamma_j * \hat{x}_{ij} + \beta_j$ (γ, β learnable per feature). At inference: use running statistics (EMA of μ, σ^2 computed during training). Importance: (1) Reduces internal covariate shift (distribution of inputs to each layer changes as parameters update — BN stabilises this). (2) Allows higher learning rates → faster convergence. (3) Less sensitive to weight initialisation. (4) Mild regularisation effect (batch statistics introduce noise). (5) Gradient flow: normalised activations → avoid saturation → better gradient flow. Limitation: depends on batch size (fails for batch=1); Layer Norm used instead for Transformers.

Q24. Explain real-world applications of deep learning in at least five domains. [5 Marks]

Answer:

1. Computer Vision: Object detection (YOLOv8 achieves real-time detection). Medical imaging (retinal disease, cancer detection). Autonomous vehicles: lane detection, pedestrian recognition. Facial recognition. 2. Natural Language Processing: Machine translation (DeepL, Google Translate). Chatbots and assistants (ChatGPT, Claude). Summarisation, question answering, code generation. 3. Speech/Audio: Speech recognition (Whisper, Google Speech API). Text-to-speech (ElevenLabs, WaveNet). Music generation (AudioCraft, MusicLM). 4. Healthcare: Protein structure prediction (AlphaFold2 solved 200M+ structures). Drug discovery (molecular property prediction). ECG/EEG analysis, surgical robotics. 5. Generative AI: Image synthesis (Stable Diffusion, DALL-E 3 — text-to-image). Video generation (Sora). Code generation (GitHub Copilot). 6. Recommendation: Netflix, YouTube, TikTok use DL to personalise content. 7. Scientific Research: Climate modelling, materials discovery, particle physics event classification (CERN).

Q25. What is the learning rate? Discuss its effect on training and explain learning rate schedules. [5 Marks]

Answer:

The learning rate η determines the step size when updating parameters: $\theta \leftarrow \theta - \eta * g$. Most critical hyperparameter in deep learning. Too large: loss diverges — overshoots minimum. Too small: very slow convergence, may get stuck. Optimal: reaches minimum quickly and stably. Learning Rate Schedules: (1) Step Decay: reduce by factor γ every k epochs. $\eta_t = \eta_0 * \gamma^{\text{floor}(t/k)}$. Simple, interpretable. (2) Exponential Decay: $\eta_t = \eta_0 * \gamma^t$. Smooth continuous reduction. (3) Cosine Annealing: $\eta_t = \eta_{min} + (\eta_{max}-\eta_{min})*(1+\cos(\pi t/T))/2$. Smooth, reaches η_{min} at end. Cosine restarts add exploration. (4) Warm-up + Decay: Linear increase from 0 to η_{max} for N_{warm} steps, then cosine/step decay. Prevents instability in early training. Standard for Transformers. (5) Cyclical LR: Oscillate between η_{min} and η_{max} — helps escape saddle points, finds flatter minima. (6) ReduceLROnPlateau: Automatically reduce η when val_loss stops improving. Practical default.

Q26. Explain underfitting and overfitting. How do you detect them from learning curves? [5 Marks]

Answer:

Underfitting (High Bias): Model too simple to capture underlying patterns. Symptoms from learning curves: Training loss is high (model makes errors even on training data). Validation loss

is similarly high. Small gap between train and val loss — both are bad. Causes: model too small, insufficient training, over-regularisation, poor features. Fixes: larger model, more epochs, less regularisation, feature engineering. Overfitting (High Variance): Model memorises training set noise. Symptoms from learning curves: Training loss is very low and decreasing. Validation loss decreases initially, then increases (diverges from train loss). Large generalisation gap = val_loss - train_loss. Causes: model too complex relative to data size, insufficient regularisation. Fixes: dropout, L1/L2, data augmentation, early stopping, smaller model, more data. Good Fit: Both train and val losses decrease together. Val loss slightly above train loss (expected). Plateau at a low value. Gap is small. Early stopping: halt training at the epoch where val loss is minimum — restore checkpoint. This prevents the overfitting phase from being reached.

Q27. Describe the evolution of neural network architectures from perceptron to modern deep networks. [5 Marks]

Answer:

Perceptron (Rosenblatt, 1958): Single-layer, binary threshold activation. Output = sign(w·x + b). Limitation: only linearly separable problems. Minsky & Papert (1969) proved XOR is impossible — first AI winter. Multi-Layer Perceptron (MLP, 1980s): Multiple layers with sigmoid activations. Backpropagation (Rumelhart et al., 1986) enables training. Universal approximation theorem: one hidden layer can approximate any function. Still limited by vanishing gradients. LeNet (LeCun, 1989): First successful CNN for digit recognition. Convolutional layers + pooling + FC layers. Introduced parameter sharing and local connectivity. AlexNet (2012): Deep CNN with ReLU (vs sigmoid), Dropout, GPU training. Won ImageNet by 10%+ — launched modern DL era. VGGNet (2014): Very deep (16-19 layers) with uniform 3×3 convolutions. ResNet (He et al., 2015): Residual connections → 50-152 layers. Solves degradation problem. LSTM/GRU (1997/2014): Gated recurrent networks for sequences. Attention (2015) + Transformer (2017): Self-attention replaces recurrence. BERT, GPT series: pretrained Transformers as foundation models (2018+). Vision Transformer/ViT (2020): Attention for images — patches as tokens.

Q28. What is a computational graph? Explain dynamic vs static graphs. [5 Marks]

Answer:

A computational graph is a directed acyclic graph (DAG) where: Nodes represent operations (matmul, relu, softmax, loss). Edges represent tensors (data flowing between operations). It records how outputs depend on inputs — enabling automatic differentiation. Forward pass: data flows from input to loss. Backward pass: traverse graph in reverse, multiplying local gradients (chain rule) to compute parameter gradients. Static Graphs (TensorFlow 1.x / tf.Graph): Graph defined once before execution. Graph can be compiled, optimised (op fusion, constant folding), serialised. Execution: feed data into compiled graph. Advantage: fast inference, deployable. Disadvantage: difficult to debug, requires explicit graph construction code. Dynamic Graphs (PyTorch / TensorFlow 2.x Eager): Graph built on-the-fly during forward pass with each call. Python control flow (if/for) works natively — flexible architecture design. Easy to debug (standard Python debuggers, print statements). Disadvantage: some optimisations unavailable. PyTorch: torch.autograd records operations on tensors with requires_grad=True. loss.backward() triggers reverse traversal. Each tensor's .grad populated. torch.no_grad(): disable graph building during inference (saves memory).

Q29. Explain the concept of model capacity and its relationship to overfitting and underfitting. [5 Marks]

Answer:

Model capacity refers to the set of functions a model can represent — how complex a mapping it can learn. High capacity: more parameters, deeper network, more expressive. Low capacity: fewer parameters, shallower network, limited expressiveness. Factors affecting capacity: Number of layers (depth): each layer can represent more complex transformations. Layer width (neurons per layer): more neurons = more linear regions (for ReLU networks). Activation function choice: ReLU networks have piecewise linear decision boundaries. Architectural structure: CNNs constrain capacity via weight sharing; Transformers have high capacity. Relationship to Fitting: Low capacity (small model) → underfitting. Cannot represent the true function. High training AND validation error. Optimal capacity: Just enough to fit true underlying pattern. Low training and validation error — small gap. Too high capacity (large model) → overfitting. Memorises training noise. Low training error, high validation error. Universal Approximation Theorem: An MLP with one sufficiently wide hidden layer can approximate any continuous function to arbitrary precision — given enough neurons. Depth theorem: Deeper networks can represent exponentially more functions per parameter than shallow networks (theoretical justification for depth). Modern DL observation: Overparameterised models (more parameters than training examples) can still generalise with proper regularisation and training (implicit regularisation via SGD noise and architecture).

Q30. What is stochastic gradient descent (SGD)? Explain its role in training deep networks and discuss common challenges. **[5 Marks]**

Answer:

SGD updates model parameters using the gradient computed from a single randomly chosen training example (or mini-batch) per step: $\theta \leftarrow \theta - \eta * \nabla L(\theta; x_i, y_i)$. Role in Training: The fundamental mechanism for optimising neural network loss functions. Without SGD (or its variants), deep learning would not be computationally feasible. Generates a noisy estimate of the true gradient — enables frequent parameter updates even for million-sample datasets. Noise as regularisation: stochasticity prevents convergence to sharp minima (which overfit) — tends to find flat minima (which generalise better). Challenges: (1) Choosing learning rate: too high → diverge; too low → slow. Requires scheduling. (2) Saddle points: in high-dim spaces, most critical points are saddle points (gradient=0 but not minimum). SGD noise helps escape. (3) Ill-conditioned loss surfaces: narrow ravines with different curvatures in different directions → slow convergence. Momentum/Adam addresses this. (4) Vanishing/exploding gradients: deep networks suffer from gradient instability. Gradient clipping, BatchNorm, ResNets help. (5) Local minima: in deep networks, local minima are often comparable to global minimum — not a major practical problem. (6) Batch size selection: small batches → noisy but regularising; large batches → stable but may generalise worse. (7) Non-stationary data: distribution shift between batches can cause instability.

— SECTION C: 2-MARK QUESTIONS —

Q36. Define a scalar and a vector in the context of deep learning. [2 Marks]

Answer:

Scalar: a single number (rank-0 tensor) — e.g., loss=0.47, learning rate=0.001. Vector: a 1D ordered array of scalars (rank-1 tensor) — e.g., feature vector (784,), weight vector.

Q37. What is the role of the learning rate in gradient descent? [2 Marks]

Answer:

The learning rate η controls the step size of parameter updates: θ←θ-η*g. Too large → divergence. Too small → slow convergence. It is the most critical hyperparameter in deep learning.

Q38. Define backpropagation. [2 Marks]

Answer:

Backpropagation computes gradients of the loss with respect to all model parameters by applying the chain rule of calculus backwards through the computational graph — from output layer to input layer.

Q39. What is the dying ReLU problem? [2 Marks]

Answer:

Neurons with ReLU activation can become permanently inactive: if the pre-activation z<0 for all inputs, ReLU outputs 0 and its gradient is 0 — the neuron never updates. Fixed by Leaky ReLU (small slope αx for x<0).

Q40. What is an epoch in neural network training? [2 Marks]

Answer:

One epoch is one complete pass through the entire training dataset. Training typically runs for many epochs (10-1000+). One epoch = ceil(n/batch_size) mini-batch gradient steps.

Q41. Define overfitting. [2 Marks]

Answer:

Overfitting occurs when a model learns training data noise/memorises examples, achieving low training loss but high validation/test loss — failing to generalise to unseen data.

Q42. What is a loss function? [2 Marks]

Answer:

A loss function (objective function) measures the discrepancy between model predictions and true labels. The training process minimises it. Examples: MSE (regression), Cross-Entropy (classification).

Q43. What is the purpose of the softmax function? [2 Marks]

Answer:

Softmax converts raw output logits z into a probability distribution over K classes: softmax(z)_k = e^(z_k)/Σe^(z_j). All outputs ∈(0,1) and sum to 1. Used as the final layer in multiclass classification.

Q44. Define mini-batch gradient descent. [2 Marks]

Answer:

Mini-batch GD computes the gradient over a small subset (batch) of B training examples per update: θ←θ-η*(1/B)*Σ∇L. Standard practice in deep learning — balances GPU efficiency with gradient noise.

Q45. What is the difference between a shallow and deep neural network? [2 Marks]

Answer:

Shallow network: 1-2 hidden layers. Can approximate any function but may require exponentially more neurons than a deep network. Deep network: many hidden layers (3-1000+). Learns hierarchical representations; more expressive per parameter.

Q46. What is tensor broadcasting? [2 Marks]

Answer:

Broadcasting allows tensors of different shapes to be combined element-wise by implicitly expanding dimensions of size 1. E.g., bias (d,) added to activations (B, d): bias broadcast to (B, d) without copying memory.

Q47. Define the term 'activation function'. [2 Marks]

Answer:

An activation function introduces non-linearity between layers of a neural network. Without it, any deep network collapses to a single linear transformation. Examples: ReLU, sigmoid, tanh, GELU, softmax.

Q48. What is weight initialisation and why does it matter? [2 Marks]

Answer:

Weight initialisation sets the starting values of model parameters before training. Poor initialisation → vanishing/exploding gradients. He init: W~N(0,√(2/fan_in)) for ReLU. Xavier/Glorot: W~N(0,√(2/(fan_in+fan_out))) for tanh/sigmoid.

Q49. What is the generalisation gap? [2 Marks]

Answer:

Generalisation gap = validation_loss - training_loss. A small gap indicates good generalisation. A large gap indicates overfitting. A gap near zero (with high loss on both) indicates underfitting.

Q50. What is Adam optimiser's main advantage over SGD? [2 Marks]

Answer:

Adam adapts the learning rate per parameter using estimates of first and second gradient moments. This means it requires less manual LR tuning, handles sparse gradients well, and converges faster than vanilla SGD on most tasks.

Quick Reference Summary Table

Topic	Unit	Key Formula / Concept	Use Case
Scalars→Tensors	I	Rank 0–5 tensors	All DL data representation
Deep vs ML	I	End-to-end feature learning	Image, speech, NLP
Broadcasting	I	(m,1)+(1,n)→(m,n)	Batch operations
Derivative	II	f'(x) = lim[f(x+h)-f(x)]/h	Gradient computation
Chain Rule	II	∂L/∂x = ∂L/∂z * ∂z/∂x	Backpropagation
Backprop	II	δ_l = W_{l+1}ᵀδ_{l+1} ⊙ σ'(z_l)	Training all layers
Mini-batch SGD	II	θ←θ-η*(1/B)*Σg	Standard training
Adam	II	θ←θ-η*m̂/(√v̂+ε)	Default optimizer
ReLU	II	max(0,x)	Hidden layers
Softmax	II	e^z_k/Σe^z_j	Multiclass output
Cross-Entropy	II	-(1/n)Σlog(ŷ_{correct})	Classification loss
MSE	II	(1/n)Σ(y-ŷ)²	Regression loss
BatchNorm	II	x̂=(x-μ)/σ; y=γx̂+β	Training stability
Dropout	II	zero w.p. p, scale 1/(1-p)	Regularisation
Overfitting	II	val_loss >> train_loss	Needs regularisation

— End of Study Material —